

Planning Doc

- **List tasks and user stories**

1. Users can sign in or sign up to the messaging system and receive a JWT, which is used to authenticate protected requests automatically. They can send and retrieve messages via a secure GraphQL endpoint and maintain a WebSocket connection for real-time updates.
2. Users can submit tabular revenue data to an endpoint and receive structured data, an Excel file URL, analysis (including possible image-based analysis), and recommendations. Similarly, they can upload feedback lists, bill images, or trigger Instagram data fetching (e.g., post comments) to get enriched data and insights.

- **System architecture overview**

1. [Messaging System](#)
2. [Data Extraction & Analysis](#)
3. [Instagram Comments](#)

- **API endpoints design**

1. [User Management](#)
2. [Document Processing](#)
3. [Feedback Processing](#)
4. [Social Media Integration](#)
5. [GraphQL Endpoints](#) and [Websocket](#)

- **Data model description:**

1. [User Model](#)
2. [Message Model](#)
3. [Feedback Model](#) (used for Instagram Comments).

- **Assumptions and Decisions:** [Technical Decisions](#)

1. The messaging system (with its frontend) operates independently from the data extraction and analysis pipeline.
2. All data extraction (from bills, revenue tables, and feedback) is done locally using open-source OCR and table extractors — no LLMs, vLLMs, or third-party APIs involved.
3. The pipeline supports structured analysis of predefined formats and integrates the Instagram Graph API solely for comment analysis, with an extensible design for future data types.
4. Data Models are made only for those endpoints where the data persisted to database.(ex: Users data, Message Data, Comment)

Appendix

1. User Management:

POST /create-user/: Create a new user with Clerk authentication

GET /users/me: Get current user information

GET /users: List all users with pagination

2. Document Processing:

POST /bill-receipt-enrichment: Process multiple receipt images, extract data, and analyze

Input: Multiple file uploads

Output: Structured data and analysis of bills

POST /tabular-record: Process tabular data from uploaded files

Input: File upload

Output: Structured data, analysis, and recommendations

3. Feedback Processing:

POST /feedback-enrichment: Process and analyze multiple feedback items

Input: List of feedback objects

Output: Enriched feedback data with analysis

4. Social Media Integration

GET /comment-persist-analyze: Fetch Instagram comments, analyze sentiment, and store in database

Output: Processed comments with analysis

5. GraphQL Endpoints

Queries

- **messages(receiverId)**: Get messages between current user and specified receiver

Mutations

- **sendMessage(receiverId, content)**: Send a message to another user

6. WebSocket Endpoints

/ws/{user_id}: Real-time messaging connection for a specific user

7. User Model:

clerk_id: External authentication ID from Clerk

fullName: User's full name

publicEmail: User's email address

phoneNumber: User's phone number

created_at: Timestamp of user creation

8. Message Model:

id: Unique identifier

content: Message text content

sender_id: ID of the sending user

receiver_id: ID of the receiving user

created_at: Timestamp of message creation

is_read: Boolean indicating if message has been read

9. Feedback Model (For Instagram Comments):

feedback: Text content of the comment

sentiment: Sentiment analysis result

sentiment_label: Categorized sentiment (positive, negative, neutral)

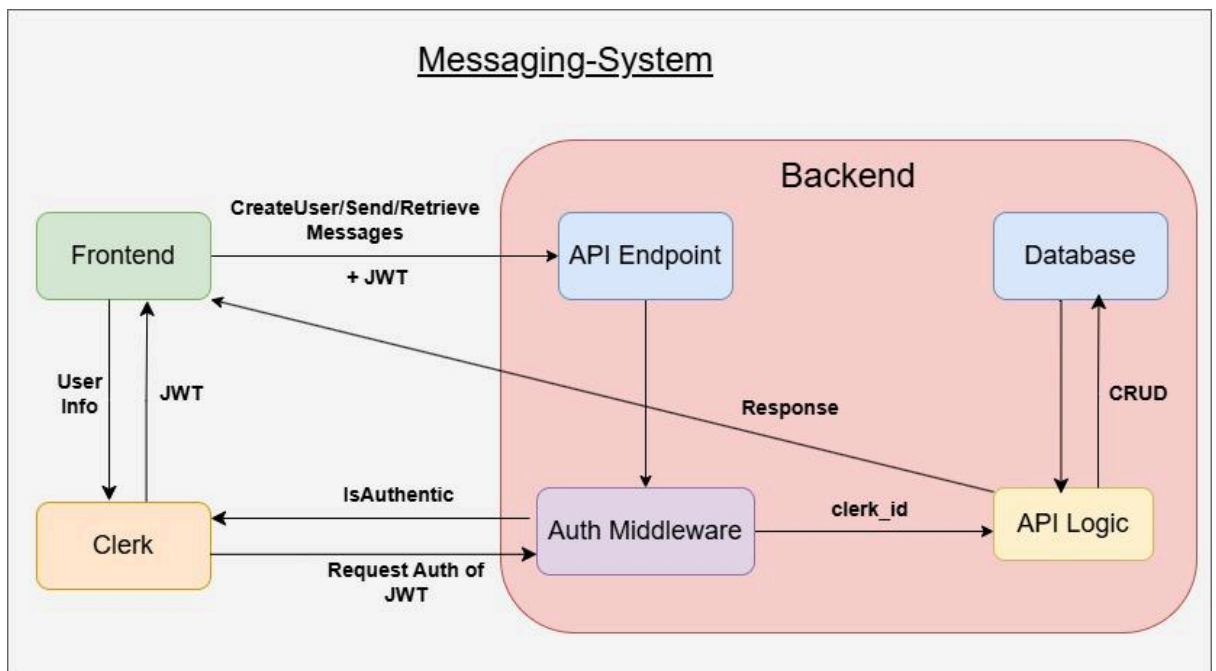
word_count: Number of words in feedback

adjectives: List of adjectives extracted from feedback

10. TechStack:

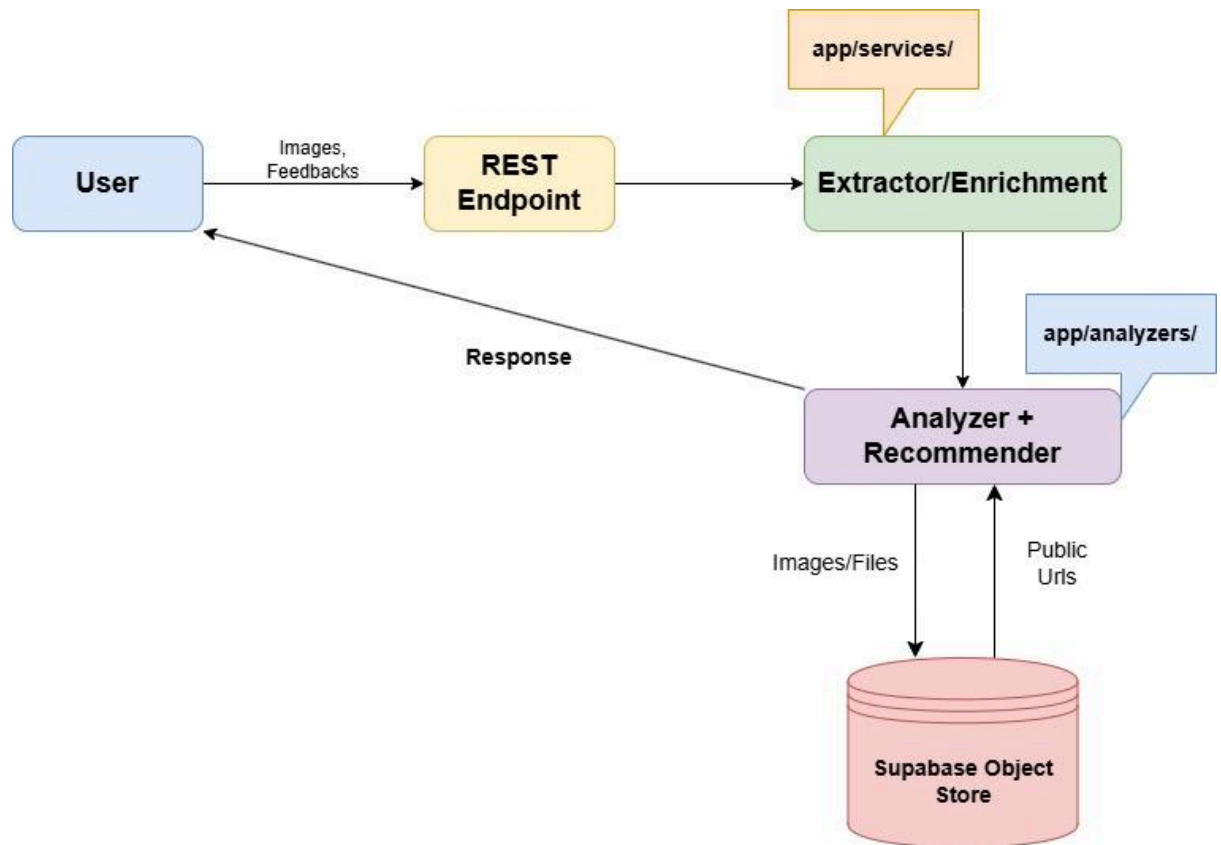
- **Backend:** Python + FastAPI + strawberry(for graphql)
- **OCR:** TesseractOCR/EasyOCR/PaddleOCR (Has to be tested).
- **Frontend:** React+Vite
- **Database:** Supabase + File Storage of Supabase.

11. Messaging System



Websocket is also implemented later.

12. Data-Extraction & Analysis:



13. Instagram Comments

